Lesson notes · for you

Read a tool call

AI agents · Upper secondary · Lesson 1

Time
20 min

Materials
Your phone or a school computer, for the code steps; Paper and a pen

Learning goals

- Tell a plan, a tool call and an answer apart.
- Split a call into its name and details in Python.
- Handle a broken call without crashing.

Activity: Activity: Call, answer or broken?

1. Warm-up (3 min): Look at weather{"place":"Kafi","day":"tomorrow"}. Circle the tool's name and underline the JSON.
2. Write (7 min): Write five lines a model might produce: two tool calls (using made-up places such as Kafi and Dala), two answers and one broken call with a bracket missing.
3. Swap (5 min): Swap with a friend. They write "call", "answer" or "broken" beside each line, and the tool's name beside each call.
4. Code (5 min): Type your lines into the code step and run parse_call on each. Did it agree with your friend?

Use only made-up places and people, such as Kafi, Dala and class 6B. You can do the first three parts with paper only.

Discussion

- Why does the model never run a tool itself?
- What should an agent do with a broken call?
- Which details would you check after json.loads?

Check yourself

- Write one tool call, one answer and one broken call, and label each.

Try it this week
On paper, write five lines an agent might produce: some tool calls, some answers and one broken call. Swap with a friend: they mark each line as a call, an answer or broken, and name the tool in every call.

Worksheet

Read a tool call

AI agents · Upper secondary · Lesson 1

Name _____ Class _____ Date _____

1 **The model wrote "Tomorrow in Kafi: sunny, high 35°C." What should parse_call return?**

◯ None: it's an answer, not a call ◯ A tool named "Tomorrow"

◯ A ValueError that stops the agent

2 **Put the lines in order so parse_call splits a call into its name and its details.**

Number the lines in order (1, 2, 3…). Cross out any line that doesn't belong.

☐ `args = json.loads(text[:i])`

☐ `return None`

☐ `i = text.find("{")`

☐ `name = text[i:]`

☐ `name = text[:i]`

☐ `import json`

☐ `return name, args`

☐ `def parse_call(text):`

☐ `if i == -1:`

☐ `args = json.loads(text[i:])`

3 **What does this program prints?**

```
call = 'weather{"place":"Kafi"}'
i = call.find("{")
print(call[:i])
```

◯ weather ◯ Kafi ◯ place

4 **Finish parse_call(text): return (name, args) for a tool call, and None for an answer (no curly bracket) or a broken call (json.loads raises a ValueError).**

```
import json

def parse_call(text):
    i = text.find("{")
    # Return (name, args), or None
    return None
```

Write your program:

5 **This parse_call gives the wrong tool name. Which line is the mistake?**

Circle the line with the mistake. Write it correctly:

```
1  def parse_call(text):
2      i = text.find("{")
3      if i == -1:
4          return None
5      name = text[:i + 1]
6      args = json.loads(text[i:])
7      return name, args
```

✎ **A model's text decides nothing until your code reads it. Where else does a program have to read text it didn't write, and what could go wrong?**